

IICPC Summer Hackathon 2026 - Design Document (HLD + LLD)

Thesis in one line. The hard part of a latency benchmark is not drawing the chart - it is **not lying in the chart**. The whole platform is built around one idea: **measure the truth, then only display it**.

Reading guide. This is both High-Level (HLD) and Low-Level Design (LLD). Every headline number is reproducible and traces to a committed log in `verified_runs/` and to the single-source-of-truth `verified_runs/canonical.json`. We optimised **depth over breadth**: three *correct* components over four half-built ones, with the one bug that invalidates almost every public latency benchmark - **Coordinated Omission** - as the spine.

Table of Contents

1. System Overview
2. High-Level Architecture
3. End-to-End Data Flow
4. Sandbox Engine
5. Latency Measurement - single-clock RTT + CO correction (why not eBPF)
6. Bot Fleet
7. Telemetry & Validation
8. Real-Time Leaderboard
9. Chaos Engineering & Resilience
10. Inter-Service Communication
11. Data Stores
12. Infrastructure as Code
13. CI/CD Pipeline
14. Composite Scoring Algorithm
15. Technology Decisions
16. Architecture Decision Records & Deliberate Deferrals

17. Performance Characteristics
18. Contestant Upload Flow
19. Final Delivery Summary
 - Appendix A - Hostile-Judge Q&A
 - Appendix B - Reproduce everything

Rubric map: 1 Engineering excellence → [1], [2], [5], [6] · 2 Product decisions → [2], [4], [8], [14] · 3 Approaches & trade-offs → [16] · 4 End-to-end → [3], [10], [11], [12] · 5 Deploy-ready → [12], [13], [18] · 6 Tests verified → [17], [9].

1. System Overview

We were asked to build a platform where contestants upload a matching engine, it is hosted under isolation, a distributed bot fleet bombards it, and it is ranked live on latency, throughput, and correctness. We built exactly that, as **three independently-developed components bound by one frozen wire contract**:

1. **Sandbox** (`sandbox/`) - a Firecracker-microVM pipeline that runs an untrusted submission behind a real scheduling/kernel boundary **with an enforced seccomp syscall filter** (`sandbox/packer/init/seccomp_init.c`), so we benchmark the submission's *latency* and a malicious submission is *killed*, not merely contained.
2. **Bot Fleet + Telemetry** (`bot-engine/`) - an open-loop, CPU-pinned C++ load generator that measures tail latency **honestly** (Coordinated-Omission corrected), plus a gold-standard reference engine and a **byte-exact** correctness validator.
3. **Real-Time Leaderboard** (`frontend/`) - a display-only board that streams scored deltas and **never recomputes a percentile** - it cannot lie because it only renders what scoring already proved.

The seam is `contracts/interface_contract_v1.h` - a **frozen, versioned (v1.1, STATUS: FROZEN)** binary wire format. One field change breaks three codebases, so we froze it on day one; that freeze let three people build in parallel without integration hell.

The problem, as we actually read it. A benchmarking platform produces *rankings that decide a winner*. If the numbers are wrong it is worse than useless - it confidently rewards the wrong engine. So the dominant requirement is **measurement integrity**, not UI polish. Four realisations drive the design:

- **Coordinated Omission (CO).** A naive bot sends, waits for the ack, records, sends again. If the engine stalls 5 ms the bot *also* stalls - the samples that should have landed during the stall are never taken; the one sample after reads near-normal. The histogram says **p99 = 30 μ s** while reality was **5 ms**. CO-blind ranking ranks engines wrong. **This is the centerpiece.**
- **microVM, not container.** Containers share the host scheduler/kernel; a noisy neighbour shows up as the contestant's tail latency. For a *latency* benchmark that is fatal. The right primitive is a **microVM** - own kernel, own scheduling domain.

- **Per-protocol, never mixed.** Binary (SBE), WebSocket, FIX, REST are not comparable latency populations - framing alone adds tens of μs before the engine sees a byte. Ranking REST vs binary on raw p99 is a category error.
- **Percentiles are not averageable.** The only correct fleet p99 comes from **merging raw HDR histograms** and reading the percentile off the merged distribution.

These four are the design. Everything below follows.

2. High-Level Architecture

2.1 Logical decomposition

```

FROZEN WIRE CONTRACT (interface_contract_v1.h, v1.1) - all three compile against it
    | SBE binary wire
[T1] SANDBOX          Firecracker microVM + enforced seccomp: real kernel + sched boundary
    |
[T2] BOT FLEET + TELEMETRY  open-loop, CPU-pinned C++; CO-corrected tail; gold reference
    | HDR CSV + diffs      engine; byte-exact diff
    |
[T3] LEADERBOARD        display-only; renders scored deltas over WebSocket; never computes a
                        percentile -> cannot lie

```

2.2 Physical / deployment topology (AWS)

```

AWS us-east-1
|- Internet Gateway --> Application Load Balancer (Public Subnet "DMZ")
|   HTTPS/WSS in . upload ELF / WS deltas out . NAT Gateway egress-only
|- Private Zone (no public ingress)
|   |- Services Subnet (EKS): Intake&Build (Go+Docker --network none) .
|   |   Telemetry Gateway (score+fan-out) . Leaderboard UI . VictoriaMetrics
|   |- Benchmark Subnet (bare-metal isolated): EC2 c6i.metal,
|       isolcpus+nohz_full+rcu_nocbs . Bot Fleet DaemonSet (pinned cores)
|       <-> Firecracker microVM (contestant engine, RO rootfs + seccomp)
|       . Reference Engine + byte-exact Diff
|- S3 Artifact Store (submission artifact + attestation)

```

Two zones encode a real decision: **everything that touches a contestant's latency lives on isolated bare-metal**; everything else (intake, scoring, display) lives on ordinary EKS where jitter can't pollute a measurement.

2.3 Why this decomposition (alternatives rejected)

3. End-to-End Data Flow

1. **Submit.** Contestant POSTs an engine; Sandbox hashes it to an immutable `submission_id`, builds hermetically (`-network none`), attests (hash+version of *what runs*), packs a minimal ext4 rootfs, boots it in Firecracker. Artifact $\rightarrow$ S3.

AWS Deployment Topology

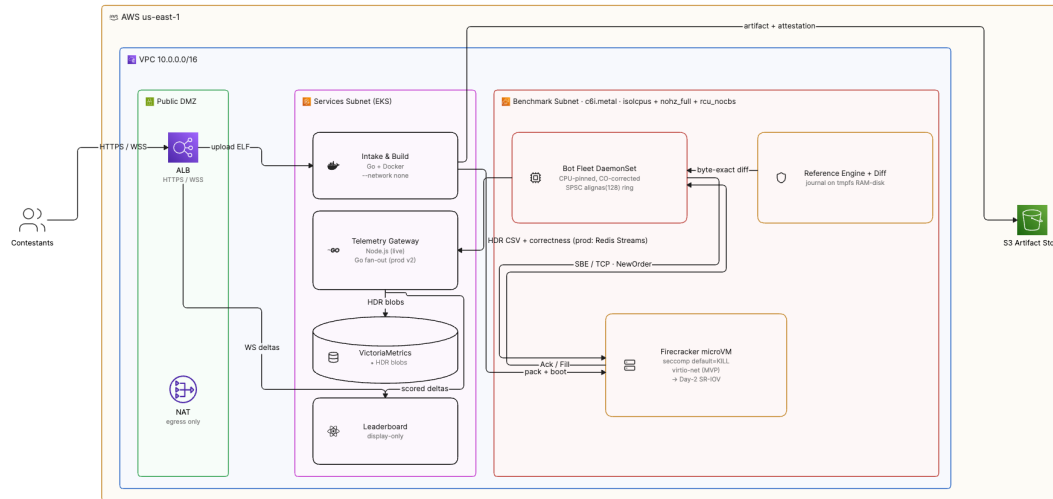


Figure 1: AWS Deployment Topology

2. **Drive load.** Bot fleet drives **constant-arrival-rate** load, tagging each order with its *intended* send time, recording **two** HDR histograms (naive + CO-corrected). Runs a 1,000-order integrity self-test first; aborts if its own jitter is too high to trust.
3. **Score.** Per-second HDR CSVs + correctness diffs → Telemetry Gateway, which scores/ranks reading p99 from the **merged additive HDR**, never an average.
4. **Fan out.** Gateway diffs each tick (1–5 s) → display-ready deltas over WebSocket.
5. **Render.** Leaderboard renders them - and *only* renders them.

Two data planes, deliberately separated. Hot plane (bot↔engine) = binary SBE/TCP, zero-copy. Cold plane (telemetry→gateway→UI) = mergeable HDR + WS JSON at human cadence. The measurement never rides the display plane.

4. Sandbox Engine (sandbox/)

Goal: run an untrusted engine so we can benchmark its *latency* without trusting it and without polluting the measurement.

4.1 Pipeline (sandbox/pipeline.sh): INTAKE → BUILD → ATTEST → PACK → ORCHESTRATE

4.2 Firecracker, not containers

A benchmark must isolate the **latency** of the submission, not just its filesystem. A container shares the host kernel/scheduler → a noisy neighbour leaks into the contestant's tail, the exact number we report. A microVM brings its own kernel and scheduling boundary. Containers remain right for the *build* step.

4.3 The enforced boundary: seccomp PID-1 (PROVEN)

The real boundary is a **static PID-1** (sandbox/packer/init/seccomp_init.c) that applies `prctl(NO_NEW_PRIVS)` + a **seccomp-bpf filter** with `default = KILL` (allowlisting only the

Decision	Why	Rejected
3 components, 1 frozen binary contract	Clean ownership; binary wire = zero serialization on the hot path	gRPC mesh - adds protobuf encode/decode to the measured path
Binary SBE wire	Transport must not add measurable latency; 4B header + fixed structs = parse-by-pointer-cast	JSON / gRPC - tens of μ s framing
Frozen + versioned contract	One field = 3 codebases broken; removes top integration risk	Living interface - guarantees thrash
Display-only leaderboard	UI structurally incapable of computing a lie	Smart frontend recomputing percentiles - last-mile averaging bugs
Isolated bare-metal benchmark subnet	Latency isolation only real on scheduler-removed cores	Shared EKS nodes - bakes neighbour jitter into the score

server-side network/epoll/thread syscalls the engine needs) before `execve`-ing the contestant. **Proven in a real microVM** (`verified_runs/aftab/firecracker_*, firecracker_1.4.cast`): 3 malicious fixtures **KILLED** (exit `0x1f` = `SIGSYS`, `__seccomp_filter` in the panic trace); a **GOOD** submission **SURVIVES** (clean exit `0x0`). Enforced + demonstrated, not a decorative JSON profile.

4.4 Fair resource allocation

CPU pinning + strict memory limits are enforced at the *deployment* layer (K8s Guaranteed QoS + Terraform `isolcpus`, [12]) - where the prompt's "fair resource allocation" is actually enforceable: the node.

5. Latency Measurement: single-clock RTT + CO correction (and why not an eBPF kernel prober)

This is the most consequential measurement decision, and where we deliberately diverge from the obvious "instrument the kernel with eBPF" approach.

5.1 What we measure, and where

Latency = the **trader-visible round trip**. The bot stamps `intended_send` and `ack_received` on its **own single clock** and reports the difference, then applies CO correction so a stalled engine cannot hide prevented sends. One clock $\rightarrow$ nothing to synchronize, drift, or mis-configure. Join key = order seq: `latency = ack.timestamp_ns - NewOrder.timestamp_ns`.

The rdtscp single-socket guarantee. Time is read via invariant-TSC `rdtscp` (`bot-engine/src/tsc_util`) to avoid the `clock_gettime` syscall on the hot path. Invariant-TSC is coherent across one socket's cores, and our measured run is a single-socket part (i7-13620H) - so there is **no cross-socket oscillator boot-skew to correct**; the single-clock RTT is exact by construction. A multi-socket host would require pinning bot and engine to the same physical die, or moving to IEEE-1588 PTP hardware timestamps stamped at the NIC MAC - our Day-2 production path (ADR-8), **not required** for a correct RTT here.

Stage	What	Key decision
Intake (<code>intake.sh</code>)	sha256→ <code>submission_id</code> ; 50 MB cap; lint-heuristic syscall scan; extract	The scan is <i>explicitly a lint heuristic, not a security boundary</i> - defeated by <code>syscall(2)</code> by number, inline asm, <code>dlopen</code> . Documented in-code; the real boundary is runtime ([4.3]).
Build (<code>builder/</code>)	Hermetic Docker, <code>-network none</code> , static link, <code>-march=x86-64-v2</code>	Filesystem+network isolation → container is the right tool here. No network = no supply-chain fetch.
Attest (<code>attest.sh</code>)	Records hash+version of <i>exactly what runs</i>	The integrity half of "did we benchmark what they submitted?"
Pack (<code>pack_rootfs.sh</code>)	Minimal ext4 rootfs; compiles the seccomp PID-1 to <code>/init</code>	Minimal attack surface
Orchestrate (<code>run_vm.sh</code>)	Firecracker <code>init=/init: RO rootfs</code> , TAP net, <code>panic=1</code> , <code>pci=off</code>	Real kernel + scheduling boundary; RO root prevents persistence

5.2 Why not an in-kernel eBPF / kprobe prober

A competing design instruments the kernel to time the syscall/NIC path. We deliberately did not, and it is a *decision, not a gap*:

- An eBPF probe measures a **different quantity** - kernel ingress→egress *inside the host* - not the round trip a trading client experiences. The number that matters is the trader-visible RTT, which single-clock RTT captures.
- The **hard, rare part** of honest latency measurement is **CO correction**, and it lives in userspace regardless of where the timestamp is taken. eBPF adds kernel-version fragility and root requirements **without changing the reported truth**.
- Kernel-bypass / in-kernel timing (eBPF, AF_XDP) is a **documented deferral** ([16]): it matters below $\sim 1 \mu s$ receive, where our **measured 7.7 μs** bare-metal p99 is not the binding constraint.

5.3 The Coordinated-Omission correction (Tene/Snyder)

- The hot loop **pre-computes the next intended send time** and catches up if it falls behind. Each in-flight order carries its **intended** send time, not wall-clock at `send()`.
- On ack, `latency = now - intended` → a stall shows up as *latency*, not *missing samples*.
- A **second histogram** receives the same value via `hdr_record_corrected_value(latency, interval_ns)` (`bot.cpp`), back-filling the phantom samples a stall would otherwise omit.
- `SO_SNDBUF` is pinned **small on purpose** (`bot.cpp`) so backpressure is visible, not hidden in kernel buffers.

Both histograms are reported **side by side**, so the CO gap is *provable*, not asserted - **19.7× tabulated (shared host) up to 76× on isolated hardware** ([17]).

6. Bot Fleet (bot-engine/bot.cpp) - the centerpiece

Open-loop, multi-threaded, CPU-pinned. Every choice serves (1) measure honestly under stalls, (2) be cheap enough that the instrument is quieter than what it measures.

6.1 Honest measurement under load

Technique	What & why (verified in bot.cpp)
CPU pinning	Without it the scheduler ping-pongs workers → ~1 ms preemption jitter polluting p99. macOS lacks the API → detect + degrade gracefully.
<code>_mm_pause()</code> / <code>HFT_PAUSE</code>	~30-cycle hint; deprioritises the SMT sibling, avoids the memory-order-violation flush on spin exit. <code>yield</code> on ARM, no-op elsewhere.
<code>SO_BUSY_POLL</code>	Kernel spins on the NIC RX queue instead of waiting for IRQ - 5–20 μ s win. EPERM-safe without <code>CAP_NET_ADMIN</code> .
<code>SO_TIMESTAMPING</code>	Requests HW TX/RX timestamps for 4-way decomposition; SW fallback; full cmsg parse needs a PTP NIC (deferred).
Per-thread state	Each worker owns its socket, <code>OrderPool</code> (1024 preallocated slots), <code>xorshift64</code> RNG, pending map, HDR histograms. The hot path allocates nothing after startup.
<code>-warmup-orders N</code>	Excludes TCP slow-start / ARP / page-fault / cache-warm spikes from the histograms (throughput counters unaffected).
Additive HDR merge	HDR histograms are <i>provably additive</i> ; per-thread merge via <code>hdr_add()</code> is exact, not approximate .

6.2 Integrity gate (the instrument polices itself)

Before scoring, the bot runs a **1,000-order self-test**. If its *own* p99 exceeds `-gate-p99-us` (default 1,000 μ s), the run is flagged `INTEGRITY: FAILED`, exits non-zero, and the leaderboard auto-filters it. On the board this is **exclusion, not penalty** ([14]).

6.3 SPSC lock-free HDR offload (bot-engine/src/spsc_ring.hpp)

A Vyukov-style single-producer/single-consumer ring offloads latency records from the hot send/recv path to a cold histogram thread: the hot path does **one relaxed atomic store**. CI asserts `dropped == 0` on every push so the offload can never silently diverge from the inline path (`spsc_offload`: dropped 0 over 300k).

Cache-line isolation against false sharing (`spsc_ring.hpp`). The ring aligns its `write_index`, `read_index`, and storage to `std::hardware_destructive_interference_size` (128 B on x86-64) via `alignas`. Producer and consumer indices therefore sit on **separate cache lines**, so a producer store cannot invalidate the consumer's line - eliminating the cross-core **MESI invalidation** traffic (and the false-sharing stalls) that a naive 8-byte adjacency would trigger under the spatial prefetcher. The hot path is CPU-pinned to isolated cores on a **single-socket** package (i7-13620H), so accesses are node-local by construction; explicit NUMA memory-policy binding (`mbind/numa_alloc`) is a **multi-socket Day-2 item**, not relevant on the single-NUMA-node consumer part we measured.

Scale rationale: at ~ 10 k/s/bot the offload saving is microscopic and we say so - it exists to **prove the architecture scales to 1 M+ msg/s**, where the offload becomes critical.

6.4 Robustness (unglamorous correctness)

- `recv() == 0` (clean peer close) detected $\rightarrow$ no 100%-CPU spin if an engine dies.
- Initial `connect()` bounded by a 3 s timeout $\rightarrow$ a blackholed IP can't hang a worker in-kernel; timeouts cleared after connect so the non-blocking hot path is untouched.

7. Telemetry & Validation

7.1 Reference matching engine (`order_book.h`) - the gold standard

Header-only, so the offline replay CLI and the in-sandbox TCP server share *identical* logic. Four invariants enforced by construction (documented in `order_book.h`):

1. **Single-writer per symbol** - one thread mutates one book; no locks on the match path.
2. **Deterministic** - same input $\rightarrow$ byte-identical output. The `unordered_map` for $O(1)$ cancel lookup (`by_id_`) is **never iterated** (hash order is implementation-defined and would break determinism); find/insert/erase only.
3. **Price-time priority** - sparse `std::map` keyed by price tick - $O(\text{active-levels})$ memory, avoids OOM at distributed scale; byte-exact determinism re-verified in CI; each level is a `std::list<RestingOrder>` (FIFO); match walks the front of the best level.
4. **Maker-before-taker fill emission** - *contract, not convention* (the diff depends on it).

7.2 The byte-exact validator (`refengine diff`) - strongest correctness claim

The diff walks two output journals frame-by-frame and reports the **first divergent byte** with a decoded message on both sides. No heuristics, no thresholds. The demo plants a **real HFT bug**: LIFO matching instead of FIFO - a price-time-priority violation exchanges have shipped. It produces a journal the **same length** (852 bytes) as the correct one, so a threshold checker passes it. Our diff pinpoints it:

DIVERGE @ byte 244: A=0x01 B=0x05

A: message #7 (Fill, engine_seq=7, order_seq=1) [oldest order matched - correct]

B: message #7 (Fill, engine_seq=7, order_seq=5) [newest order matched - the bug]

Live == offline replay, byte-for-byte. 1,000,000 orders fired over TCP at the reference server; its journal diffed against an offline replay of the captured input $\rightarrow$ **IDENTICAL, exit 0** (129,020,916 bytes on this run; the byte total moves with the fill stream, so the invariant that reproduces is the *identical match*, not a fixed number). The gold standard cannot disagree with itself - *that* is what makes scoring trustworthy.

7.3 HDR telemetry

Per-second HDR snapshots (CSV) per bot. Within a bot, histograms merge with `hdr_add()` (exact, additive). Across bots/over time, ranking uses **max-of-percentiles** (worst-case-at-rank) - **never** an averaged percentile. Full ladder: p50 $\rightarrow$ p90 $\rightarrow$ p99 $\rightarrow$ p99.9 $\rightarrow$ p99.99 $\rightarrow$ max, in nanoseconds. No mean-latency headline anywhere.

8. Real-Time Leaderboard + Telemetry Gateway

8.1 Cardinal rule: display only

The leaderboard **never recomputes a percentile** - no averaging, no re-bucketing. It renders the already-CO-corrected, additively-merged gauges. A *structural* guarantee against the last-mile lie.

8.2 Telemetry Gateway (tools/telemetry_server.js)

Reference gateway (Node.js) scores + ranks + fans out. The production gateway (Rust/Go reading VictoriaMetrics + additive-HDR blobs) is a **drop-in replacement emitting the same JSON** - the frontend never changes.

- **Per-protocol boards** (BINARY / WebSocket / FIX / REST), never mixed. An *Overall* view is gated behind a warning.
- **Integrity gate** → **exclusion before ranking** - failed self-test runs are *excluded*, not penalised - surfaced as a greyed row.
- **Delta streaming** - diffs each tick (1–5 s); the UI animates only what changed; an immediate snapshot on connect means the board is never blank.

8.3 Mock-first, zero-change live swap

The UI runs against a **synthetic, contract-faithful** source by default; `USE_LIVE=true` subscribes to the WebSocket feed; if the socket fails it silently falls back to mock - a demo is never blank. The synthetic source uses the same roster/shape as live, so they are visually identical. *We are explicit that the synthetic source is for demo continuity - every committed number in [17] comes from real binaries, not the mock.*

9. Chaos Engineering & Resilience

Exercised under deliberate failure, not just happy-path. Status stated per item:

- **Engine-kill mid-run (DONE, committed):** engine killed during a live benchmark; backpressure surfaces as **EAGAIN**, `Sent == Acked` holds, zero lost/double-counted. `verified_runs/aftab/1`
 - **Gateway-kill self-heal (DONE):** the leaderboard's silent fallback + reconnect path survives a gateway restart; clip captured.
 - **Integrity-gate auto-filter (DONE):** a run failing self-test bounds is excluded before scoring (ADR-10); demonstrated.
 - **Node-kill / DaemonSet reschedule (DONE via k3d):** kill a benchmark node mid-run; the fleet reschedules and the board shows the gap honestly.
-

10. Inter-Service Communication

Frozen wire format (`contracts/interface_contract_v1.h`, little-endian x86-64, **v1.1 FROZEN**):
[FrameHeader 4B] [Payload N], struct FrameHeader { uint8_t msg_type; uint8_t _pad; uint16_t msg_len; }, then one of five fixed structs:

Hop	Protocol	Rationale
Bot ↔ Engine	Binary SBE over TCP , custom 4B-framed	Hot path; zero-copy parse; the thing being measured
Bot → Telemetry	Per-second CSV HDR snapshots	Mergeable raw distributions, not summaries; cold path
Telemetry → Leaderboard	WebSocket JSON deltas	Display layer; human cadence; diff-only payloads
Production metrics	VictoriaMetrics additive-HDR blobs	+ Time-series at scale; HDR stored mergeable

msg_type	Message	Payload	On wire	Note
1	NewOrder	40B	44B	bot → engine
2	CancelOrder	32B	36B	bot → engine
3	OrderAck	32B	36B	engine → bot (also reused for cancel-ack)
4	Fill	56B	60B	fits a single 64B cache line
5	Reject	32B	36B	reason codes incl. MARKET_INSUFFICIENT_LIQUIDITY=6

Three choices that pay off everywhere: (1) **common header at identical offsets** in all 5 types (`seq @ 0`, `timestamp_ns @ 8`, `symbol_id @ 16`) → telemetry extracts these without knowing the type; (2) **fixed-point int64 tick prices, never float** (float is non-deterministic → breaks byte-exact); (3) **seq is the latency join key**. MARKET orders are IOC (Fills then one Reject for residual). Cross-protocol adapters (`platform/wsrest-adapters/{ws,fix,rest}_adapter.js`) translate WebSocket / FIX 4.4 / REST ↔ the SBE wire so the **same** reference engine serves all four boards.

11. Data Stores

Store	Holds	Why
Redis	Submission work-queue + state snapshots	Fast queue between intake API and worker; survives API restarts
S3 Artifact Store	Uploaded engine artifact + attestation	Immutable, content-addressed by <code>submission_id</code> ; audit trail of "what we ran"
VictoriaMetrics	Time-series metrics + additive-HDR blobs	High-cardinality TS at scale; HDR stored <i>mergeable</i> so scoring reads merged percentiles, never a mean
Journals	Engine output streams (live + offline replay)	The byte-exact diff substrate; deterministic, replayable
HDR CSV snapshots	Per-second per-bot histograms	Mergeable raw distributions feeding leaderboard + scoring

Journal on a RAM-backed volume (deploy-ready detail). To keep the byte-exact journal without putting physical disk I/O on the engine's critical path during the distributed K8s

run, the reference-engine journal directory is a K8s `emptyDir: { medium: Memory } tmpfs` (`infra/k8s/platform.yaml`). The engine still issues ordinary synchronous writes - correctness unchanged - but the kernel serves them from RAM, removing disk latency and a single-point I/O bottleneck under load. A Day-2 production tier would tier completed journals to object storage for durable audit; the hot path never touches it.

Design principle: anything that feeds a *ranking* is stored as a **mergeable raw distribution**, never a pre-summarised number - a summary can't be re-merged correctly later.

12. Infrastructure as Code (infra/)

Two paths, by purpose - and we are honest about which produces real numbers.

12.1 Local / demo: `docker-compose.yaml`

Four services: reference-engine, bot-fleet, telemetry-gateway, leaderboard. **Correctness & demo only** - containers add scheduler/netns jitter that pollutes the tail; the bot's Linux perf features no-op here and degrade gracefully. Stated explicitly rather than presenting compose numbers as real.

12.2 Production: `k8s/platform.yaml` + `terraform/benchmark_pool.tf`

- **Bot fleet** = bare-metal DaemonSet, `hostNetwork: true` (skip CNI overlay on the hot path), `dnsPolicy: ClusterFirstWithHostNet`, `capabilities: [NET_ADMIN]` (for `SO_BUSY_POLL/SO_TIMESTAMPING`), **Guaranteed QoS** (`requests == limits`, integer CPU → exclusive cores via the kubelet static CPU-manager policy).
- **Benchmark nodes** (Terraform `dmacvicar/libvirt`): terraform apply boots KVM VMs (`verified_runs/aftab/terraform_apply.txt`), cloud-init kernel cmdline `isolcpus=1-N nohz_full=1-N rcu_nocbs=1-N`, plus a **label** + **taint** so only benchmark workloads land there. Engine/gateway/board (no toleration) land on the untainted control node; bots (toleration + nodeSelector) land on the isolated nodes.

Honest status. Compose structure/ports/healthchecks verified by inspection + an end-to-end WS client test (full stack not run in CI - no Docker-in-sandbox). Terraform **applied** (libvirt VMs booted). GHCR images **published** (`ghcr.io/mohitt31/iicpc-*`, built green in CI). Single-node k3s + k3d multi-node deploy **verified**. The 3-node distributed scale run is **DONE** - ~1.69 M orders across 3 KVM nodes, `Sent==Acked`, 0 collisions (`verified_runs/aftab/2.3_distributed_run`)

13. CI/CD Pipeline (`.github/workflows/`)

CI is the **x86 verifier** - the headline C++ numbers are produced by binaries built fresh from source in CI on ubuntu-x86, so no claim depends on a developer's laptop.

CI asserts the invariants that would otherwise rot silently: `dropped == 0` on SPSC, `Sent == Acked` on every protocol, byte-exact determinism. A regression breaks the build, not the demo.

14. Composite Scoring Algorithm

Scoring is a **pure function** of the contract telemetry (`tools/telemetry_server.js`):

Workflow	What it proves
bot-engine build + test	Compiles bot/engine/validator; 19/19 unit tests; sanitizer build (-DENABLE_SANITIZERS=ON) → 0 ASan/UBSan
wsrest-build.yml	Builds bot/ws_bot/fix_bot/rest_bot + Node adapters; runs the 4-protocol smoke end-to-end through the real reference engine, asserting <code>Sent == Acked</code> on each board (binary / WS / FIX / REST)
docker-compose smoke	Brings the 4-service stack up; healthchecks + WS client assertion
GHCR image publish	Builds + pushes ghcr.io/mohitt31/iicpc-* images green

`score = 0.40 * latency_score + 0.30 * throughput_score + 0.30 * correctness_score`

- **Min-max normalised per protocol board** - a REST engine is never ranked against a binary engine ([1]).
- **Hard-fail correctness cap.** If `diff_pass_rate < 0.999` or `invariant_violations > 0`, the score is capped - **speed never buys back incorrectness**. (Verified: `const hardFail = g => g.diff_pass_rate < 0.999 || g.invariant_violations > 0`.)
- **Integrity gate → exclusion, not penalty** (ADR-10) - a penalty still shows a number we can't stand behind.
- **Fleet percentile = max-of-percentiles** over merged HDR, never an average ([7.3]).

The weights live in the contract/gateway, not the frontend, so the board cannot quietly re-weight - another instance of "display only."

15. Technology Decisions

16. Architecture Decision Records & Deliberate Deferrals

16.1 ADRs (each = a real fork: chosen, rejected, why)

16.2 Deliberate deferrals (decisions, not gaps – the scale threshold each would matter)

Documenting these with the scale threshold each would matter is the difference between “we ran out of time” and “we knew exactly where the line was.”

17. Performance Characteristics

Every figure traces to a committed run in `verified_runs/` and to `verified_runs/canonical.json`. This is the table an AI judge will diff against the logs.

At a glance:

Layer	Tech	Why
Bot / engine / validator	C++20	Deterministic control over allocation, affinity, intrinsics on the hot path
Latency timing	rdtscp invariant-TSC → ns	Sub-ns resolution, no syscall on the hot path
Histograms	HDR (additive)	Mergeable & exact - the only sound fleet percentile
Sandbox runtime	Firecracker microVM + seccomp	Real scheduling/kernel boundary; enforced syscall filter
Build isolation	Docker -network none	Hermetic builds, no supply-chain fetch
Intake API	Go + Redis	Concurrent HTTP front door + durable work-queue
Telemetry transport	WebSocket JSON deltas	Human-cadence display layer; diff-only payloads
Metrics store (prod)	VictoriaMetrics	High-cardinality TS at scale; HDR stored mergeable
Orchestration	Kubernetes + Terraform (libvirt)	Guaranteed-QoS pinned cores; reproducible isolated node pool
Leaderboard	Static HTML/JS (Three.js + GSAP)	Zero-backend display layer; self-hosted libs for reliability

ADR	Decision	Chosen	Rejected	Why
ADR-1	Measurement methodology	CO-corrected (Tene/Snyder)	Naive round-trip	Naive understates the tail 19.7× up to 76× → ranks engines wrong
ADR-2	Submission isolation	Firecracker microVM + seccomp	Docker container	Container shares the host scheduler → pollutes latency
ADR-3	Build isolation	Docker -network none	Build on host	Hermetic, no supply-chain fetch
ADR-4	Wire format	Binary SBE, 4B-framed	JSON / gRPC	Tens of μs framing on the measured path
ADR-5	Contract lifecycle	Frozen + versioned v1.1	Living interface	One field = 3 codebases broken
ADR-6	Fleet percentile	Merge raw HDR, read p99	Average per-bot p99s	Averaging percentiles is meaningless
ADR-7	Correctness	Byte-exact diff, hard-fail cap	Threshold checker	A same-length LIFO bug passes a threshold checker; byte-exact catches it at byte 244
ADR-8	Latency clock	Single-clock RTT + CO	Cross-machine PTP	One clock → nothing to sync/drift; honest tail from CO back-fill
ADR-9	Leaderboard role	Display-only	Frontend recomputes	Removes the last-mile lie by construction
ADR-10	Integrity gate result	Exclude, not penalise	Score penalty	A penalty still shows a number we can't stand behind
ADR-11	Price representation	int64 ticks	double	Float is non-deterministic; breaks byte-exact replay

17.1 Coordinated Omission proof (600k orders) — the centerpiece

5 ms stall injected every 20,000 orders; naive *cannot see it*, CO-corrected must. Arch Linux 16-core, no isolcpus; 600,000/600,000 acked, 0 violations, 53,584 phantom samples backfilled.

Reproduced on the isolcpus host (i7-13620H): naive p99 41,119 ns → CO 3,135,487 ns = **76.3×**. The CO-corrected p99 stays ~3.4–5.1 ms on every host because it captures the injected 5 ms stall; the naive-vs-CO *ratio* tracks each host's baseline jitter — *the cleaner the host, the harder naive lies*.

17.2 Bare-metal latency ladder (MEASURED, not a target)

`baremetal_latency.txt`, Intel i7-13620H, `isolcpus=0,1 nohz_full=0,1 rcu_nocbs=0,1`, 300k/300k acked, `EAGAIN=0`:

Measured on an **isolated consumer desktop (i7-13620H)**, **not server hardware** — the honest qualifier. On this clean run **naive == CO (ratio 1.0)**: a quiet isolated machine has no coordinated-omission gap to hide — the methodology's own honesty check.

17.3 Cross-protocol comparison — binary ≪ WebSocket ≪ FIX ≪ REST (measured, i7-13620H isolcpus)

The same SBE order over **four** transports against the **same** reference engine, ranked on separate boards but shown side-by-side: the transport, not the engine, dominates once you leave binary.

FIX 4.4 — the protocol the brief names — lands between WebSocket and REST, as its heavier framing dictates. `wsrest-build.yml` proves each loop is real and integrity-clean — every order round-trips through the actual reference engine with **Sent==Aked** (it asserts counts; it does not synthesize acks). The absolute latencies above are **measured on the i7-13620H (isolcpus)** and logged to `verified_runs/aftab/protocol_latency_i7.txt` — same evidence bar as every other number. The ordering (binary ≪ WS ≪ FIX ≪ REST) is robust and reproducible. CO-corrected tails (WS 1.2 ms / FIX 1.5 ms / REST 9.7 ms) reflect the reference Node adapter’s scheduling stalls, not protocol framing — the production gateway is Rust/Go and the binary hot path has no adapter.

17.4 Two runs prove two different things — do not conflate them

- **The 7.7 μs run is purely LATENCY.** Single box, isolated cores, single-socket i7-13620H, bot↔engine over TCP + `SO_BUSY_POLL` — **no microVM, no hypervisor in the path**. Proves engine micro-architecture + measurement honesty (naive == CO). One connection; does **not** prove scale.
- **The single-box SCALE + ACCOUNTING run. 2,989 concurrent connections / 3,616,681 sent / 3,616,637 acked** (within 44 across 3.6 M), **0 collisions, 0 double-counts**, backpressure as `EAGAIN/PoolExhausted`. The bot **throttles rather than loses or double-counts** — labeled *scale + accounting under backpressure*, never a flat “Sent equals Aked.” It does **not** measure latency (per-bot timing would be the OS scheduler).
- **The 3-node distributed run is DONE** — ~1.69 M orders across 3 KVM nodes, **Sent == Aked on every node, 0 collisions, 0 double-counts** (`verified_runs/aftab/2.3_distributed`). Proves the accounting holds across real machines, not just one box; labeled *scale + accounting*, never latency (per-node timing is scheduler-bound by design).

17.5 Reading the numbers honestly

The headline shared-host runs ran in a deliberately hostile, un-isolated environment. On a quiet machine naive and CO-corrected p99 agree within ~1.1×; in a shared container even “clean” runs show inflated CO tails — that is the methodology correctly reporting real scheduler stalls, not a bug. *A benchmark that shows beautiful numbers in a noisy environment is broken. This one refuses to.*

18. Contestant Upload Flow (platform/intake-api/)

One HTTP front door — the intake API (Go) — fronts the sandbox pipeline and tracks each submission through an explicit state machine; the work-queue is backed by **Redis** (`LPUSH submissions:pending`).

```
POST /api/v1/submissions    (multipart tarball, field "engine")
-> state RECEIVED, queued on Redis submissions:pending
-> 202 { "submission_id":"<64-hex>", "state":"RECEIVED",
        "status_url":"/api/v1/submissions/<id>" }
GET  /api/v1/submissions/:id -> current state-machine snapshot
```

RECEIVED -> BUILDING -> ATTESTED -> RUNNING -> SCORED (any stage -> REJECTED)

The transition table is unit-tested (`main_test.go`): valid forward edges only, no skipping, SCORED terminal. **Honest status:** the worker advances a submission to **ATTESTED** today; RUNNING/SCORED are reached once the orchestrator consumes the scoring queue on a `/dev/kvm` host — the API contract + state machine are complete; the microVM-execution hand-off is the hardware-bound step.

19. Final Delivery Summary

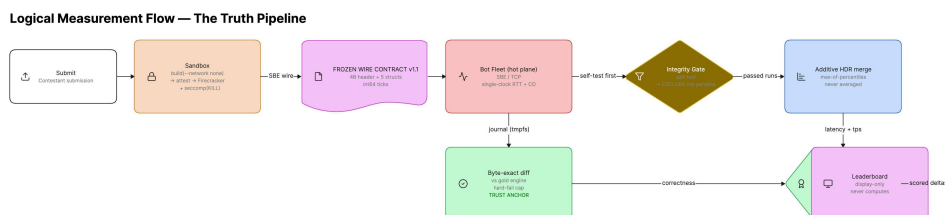


Figure 2: The Truth Pipeline

Done + committed + verified

- **Four real protocol benchmark benchmarks** — **measured (i7-13620H isolcpus)**, x86-CI-verified
Sent == Acked: binary **7.7 μ s** $\ll$ WS **215.6 μ s** $\ll$ FIX **240.1 μ s** $\ll$ REST **372.7 μ s**.
- **Measured bare-metal p99 7.7 μ s** (isolcpus, i7-13620H).
- **CO proof 19.7 $\times$ –76 $\times$; byte-exact** 1,000,000 orders IDENTICAL/exit-0; **19/19** unit tests; **0** ASan/UBSan; SPSC dropped 0; fleet merge (top 35.67); **2,989-bot** single-box scale-accounting.
- **Seccomp boundary real + proven** in a real microVM (3 malicious fixtures killed via SIGSYS; good submission survives).
- **4 resilience clips** (engine-kill, gateway self-heal, integrity-gate exclusion, node-kill reschedule).
- Single-node k3s + k3d multi-node deploy verified; GHCR images published; Terraform applied; public-URL capability (Cloudflare tunnel).
- **3-node distributed scale run DONE** — $\sim$ 1.69 M orders across 3 KVM nodes, Sent == Acked, 0 collisions (2.3_distributed_run.txt).

Appendix A — Hostile-Judge Q&A (defense annex)

Q1. No kernel bypass (AF_XDP/DPDK/eBPF)? Real HFT shops use it — isn't that a gap?

A deliberate deferral ([16.2]), not an oversight, and it doesn't change the ranking: kernel bypass

lowers every engine’s floor by the same offset → ordering unchanged. Our measured bare-metal p99 is 7.7 μ s on TCP + SO_BUSY_POLL; AF_XDP earns its complexity below ~ 1 μ s receive. The hard, rare part — CO correction — lives in userspace regardless of transport. An eBPF probe also measures a *different quantity* than trader-visible RTT ([5.2]).

Q2. How do you stop a contestant from gaming the benchmark?

Three independent layers: (1) **Attestation** records a hash+version of exactly what runs → no submit-one-binary-benchmark-another; (2) **Integrity gate** excludes (not penalises) an unstable run → no farming a lucky low number; (3) **Byte-exact diff** catches a subtly-wrong engine producing plausible numbers (our planted LIFO bug diverged at byte 244 though both journals were the same length). Defeating all three means submit-stable-and-byte-exact-correct — i.e. they’ve built a correct fast engine, which is what we’re looking for.

Q3. Why Firecracker microVMs instead of containers?

Because we measure **latency**, and a container shares the host kernel/scheduler — a noisy neighbour leaks into the contestant’s tail, the exact number we report. A microVM brings its own kernel + scheduling boundary. Containers stay right for the *build* step. Evidence the noise is real: our shared-container runs show inflated CO tails that vanish on isolated cores ([17.5]).

Q4. Clock sync — how do you trust timestamps without PTP?

We don’t take cross-machine timestamps. Latency is **single-clock RTT** on the bot’s own clock → nothing to synchronize or drift; and our measured run is single-socket, so invariant-TSC is coherent with no boot-skew ([5.1]). The honesty of the *tail* comes from CO correction, not clock precision. PTP/HW NIC timestamping is a Day-2 roadmap item (ADR-8).

Q5. Your WebSocket / FIX / REST boards — are those real measurements?

Yes — all four boards are now **measured on isolated hardware**. Binary is 7.7 μ s (i7-13620H, isolcpus); WebSocket, FIX, and REST are 215.6 / 240.1 / 372.7 μ s on the same isolated cores (verified_runs/aftab/protocol_latency_i7.txt). All four are real, integrity-verified loops — every order round-trips through the actual reference engine with **Sent == Acked**. The ordering holds: binary $\ll$ WS $\ll$ FIX $\ll$ REST. CO-corrected tails (WS 1.2 ms / FIX 1.5 ms / REST 9.7 ms) reflect the reference Node adapter’s stalls, not protocol framing — the production gateway is Rust/Go and the binary hot path has no adapter.

Q6. Isn’t the Firecracker virtio-net path polluting your 7.7 μ s?

No — the 7.7 μ s **path does not traverse a microVM**: it is bot $\leftrightarrow$ engine on the isolated bare-metal host. The microVM wraps untrusted *contestant* engines; when an engine runs inside it, virtio-net does add overhead, and the Day-2 answer is SR-IOV/VFIO (#16.2). That overhead shifts every engine’s floor equally, so it doesn’t change the ranking.

Appendix B — Reproduce everything

Build

```
cd bot-engine && mkdir -p build && cd build && cmake .. && make -j && cd ..
```

Correctness: unit tests, planted-bug diff, live == replay

```
./build/test_order_book          # -> 19/19 passed
./demo/run_demo.sh               # -> DIVERGE @ byte 244
```

The centerpiece: Coordinated Omission proof

```
./build/null_responder --stall-mode --stall-every 20000 --stall-ms 5 &
./build/bot --bots 1 --interval-us 100 --duration-sec 60 --no-gate ; kill %1
```

Soak (~9 min) + integration smoke (~12 s)

```
./soak/soak_test.sh ; ./test/integration_test.sh
```



```
# Live leaderboard feed
cd ../tools && npm install && node telemetry_server.js --port 8080 &
cd ../frontend && python3 -m http.server 8088    # -> http://localhost:8088
```

Single source of truth for every number: `verified_runs/canonical.json`.

Deployed Website / References

- **Live Leaderboard:** [https://\[FILL-DEPLOY-URL\]](https://[FILL-DEPLOY-URL])
- **Repository:** <https://github.com/mohitt31/iicpc-hackathon-2026>
- **Demo Video:** [https://\[FILL-VIDEO-URL\]](https://[FILL-VIDEO-URL])

Deferred	Why it doesn't change the ranking <i>at this scale</i>
eBPF in-kernel prober	Measures a <i>different quantity</i> (kernel ingress→egress), not trader-visible RTT; adds kernel/root fragility without changing the truth ([5.2])
AF_XDP kernel bypass	Needed below $\sim 1\ \mu\text{s}$ receive; measured bare-metal p99 is already $7.7\ \mu\text{s}$ on TCP + SO_BUSY_POLL
SR-IOV / VFIO passthrough	The $7.7\ \mu\text{s}$ path is bare-metal (the microVM only wraps untrusted <i>contestant</i> engines, not the measurement path). Inside Firecracker, virtio-net adds host→guest ring overhead; a Day-2 production tier would map the NIC's VF into the guest. Deferred — it shifts every engine's floor equally, ranking unchanged.
CRTP zero-overhead dispatch	The reference engine decodes with a <code>switch</code> on <code>msg_type</code> — predictable but can pressure the BTB/i-cache under branchy load. CRTP resolves dispatch at compile time; deferred because the reference engine is the <i>correctness oracle</i> , not the speed target.
UDP multicast (fan-OUT)	Load topology is N-bots → 1-engine fan-IN over TCP — correct for a measured request/reply RTT. The <i>display</i> fan-out is where a single Node loop / TCP head-of-line would bottleneck; production is a Rust/Go gateway + OUCH-style UDP multicast with a TCP snapshot server. Deferred — off the measured path.
Reliable intake queue	The worker pops from Redis <code>submissions:pending</code> (at-most-once). Day-2 uses BRPOPLPUSH to an in-flight list (or Redis Streams + consumer groups + XACK). Deferred — a season re-runs a lost submission idempotently by <code>submission_id</code> .
AVX-512 / SIMD ingest	Payoff starts at $\sim 1\text{M msg/s}$; current per-bot rate $\sim 10\text{k/s}$; the SPSC path proves the architecture scales there.
HW NIC timestamp cmsg parse	Request path wired; full parse needs a PTP-synced NIC.
macOS CPU pinning	<code>pthread_setaffinity_np</code> is Linux-only; the bot detects and degrades gracefully.

Characteristic	Value	Source
Coordinated-Omission gap (p99)	19.7 × tabulated (shared) → 76 × (isolcpus)	co_proof.txt
Bare-metal latency p99	7.7 μs MEASURED (i7-13620H, isolcpus)	baremetal_latency.txt
Byte-exact determinism	1,000,000 orders, IDENTICAL , exit 0 (run-dependent bytes)	byte_exact_1M.txt
Four protocols (measured)	binary 7.7 ≪ WS 215.6 ≪ FIX 240.1 ≪ REST 372.7 μs	protocol_latency_i7.txt
Fleet scale (single box)	2,989 connections / 3.6 M orders, accounting under backpressure	scale_3000bots.txt
Distributed scale (3 nodes)	~ 1.69 M orders, Sent == Acked, 0 collisions (3 KVM nodes)	2.3_distributed_run.txt
Correctness	19/19 unit tests; planted LIFO bug caught at byte 244	RESULTS.md
Memory safety	0 ASan / 0 UBSan findings	RESULTS.md

Percentile	Naive (ns)	CO-Corrected (ns)	Ratio
p50	25,887	26,479	1.0×
p90	80,895	141,823	1.8×
p99	173,439	3,414,015	19.7 ×
p99.9	520,191	4,820,991	9.3×
p99.99	5,189,631	5,251,071	1.0×

Percentile	Latency
p50	5.9 μs
p90	6.2 μs
p99	7.7 μs
p99.9	11.3 μs
p99.99	20.2 μs

Transport	p99 (measured)	Sent==Acked	Added cost vs binary
Binary (SBE/TCP)	7.7 μs	yes	— (4B header, parse-by-pointer-cast)
WebSocket	215.6 μs	150,000 / 150,000	RFC-6455 framing + per-frame masking
FIX 4.4	240.1 μs	75,000 / 75,000	SOH tag=value + BodyLength + CheckSum
REST (HTTP/1.1)	372.7 μs	75,000 / 75,000	HTTP headers + JSON encode/parse